JAVASCRIPT:

1.Lab Question: Alert, Confirm, and Prompt – Quiz Game Question Create a web page that: 1. Welcomes the user with an alert. 2. Asks the user's name using a prompt. 3. Shows a confirmation message asking if they are ready for the quiz using confirm. 4. If confirmed, asks a multiple-choice question. 5. Displays the result (correct or wrong) using alert.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Quiz Game</title>
  <script>
    // Step 1: Welcome message
    alert("Welcome to the Quiz Game!");

    // Step 2: Ask for user's name
    let userName = prompt("What is your name?");

    // Step 3: Confirmation to start quiz
    let ready = confirm("Hi " + userName + "! Are you ready for the quiz?");

    if (ready) {
      // Step 4: Ask multiple-choice question
      let answer = prompt(
        "Which is the largest planet in our Solar System?\n" +
        "A: Earth\nB: Jupiter\nC: Mars\nD: Saturn\n" +
        "Type A, B, C, or D"
      );

      // Step 5: Check answer
      if (answer && answer.toUpperCase() === "B") {
        alert("Correct!  Jupiter is the largest planet.");
      } else {
```

```
            alert("Wrong!  The correct answer is Jupiter.");
        }
    } else {
        alert("Okay " + userName + ", maybe next time!");
    }
  </script>
</head>
<body>
</body>
</html>
```

2.Lab Question: Income Tax Calculator Question: Write a JavaScript program to calculate an individual's income tax based on the following rules: • Input: • Full Name (string) • PAN Number (string, must be exactly 10 characters) • Date of Birth (Date format) • Annual Income (number)

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Income Tax Calculator</title>
</head>
<body>
<script>
  // Step 1: Input from user
  let fullName = prompt("Enter your full name:");
  let panNumber = prompt("Enter your PAN number (10 characters, uppercase):");
  let dobInput = prompt("Enter your Date of Birth (YYYY-MM-DD):");
  let annualIncome = parseFloat(prompt("Enter your Annual Income (₹):"));

  // Step 2: Validate PAN
  if (panNumber.length !== 10 || panNumber !== panNumber.toUpperCase()) {
    alert("Invalid PAN format! Must be 10 uppercase characters.");
    throw new Error("Invalid PAN");
  }
```

```
// Step 3: Calculate Age from DOB
let dob = new Date(dobInput);
let today = new Date();
let age = today.getFullYear() - dob.getFullYear();
let monthDiff = today.getMonth() - dob.getMonth();
if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < dob.getDate())) {
   age--;
}

// Step 4: Calculate Tax
let tax = 0;
if (annualIncome <= 250000) {
   tax = 0;
} else if (annualIncome <= 500000) {
   tax = (annualIncome - 250000) * 0.05;
} else if (annualIncome <= 1000000) {
   tax = (250000 * 0.05) + (annualIncome - 500000) * 0.20;
} else {
   tax = (250000 * 0.05) + (500000 * 0.20) + (annualIncome - 1000000) * 0.30;
}

// Round to nearest rupee
tax = Math.round(tax);

// Step 5: Display output
let output = `
Name: ${fullName.toUpperCase()}<br>
PAN: ${panNumber}<br>
Age: ${age} years<br>
Annual Income: ₹${annualIncome}<br>
```

```
    Calculated Tax: ₹${tax}
    `;
    document.body.innerHTML = `<h2>Income Tax Details</h2>` + output;
</script>
</body>
</html>
```

3.Lab Question: Equality & Inequality Checker Question: Write a JavaScript program that asks the user to enter two values and then compares them using: 1. == (equality) 2. === (strict equality) 3. != (inequality) 4. !== (strict inequality) Requirements: • Show the data type of each entered value. • Explain in the output whether the values are considered equal or not under each comparison. • Use String and Number conversion to demonstrate differences.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Equality & Inequality Checker</title>
</head>
<body>
<script>
    // Step 1: Get values from user
    let value1 = prompt("Enter the first value:");
    let value2 = prompt("Enter the second value:");

    // Step 2: Detect data types (initially both are strings from prompt)
    let type1 = typeof value1;
    let type2 = typeof value2;

    // Step 3: Convert second value to number (to demonstrate type difference)
    let numValue2 = Number(value2);

    // Show type change for second value
    console.log("Second value converted to number for testing:", numValue2);
```

```
    // Step 4: Comparisons
    let equalLoose = (value1 == numValue2); // equality ignoring type
    let equalStrict = (value1 === numValue2); // equality checking type
    let notEqualLoose = (value1 != numValue2); // inequality ignoring type
    let notEqualStrict = (value1 !== numValue2); // inequality checking type

    // Step 5: Display results
    document.write(`<h2>Equality & Inequality Checker</h2>`);
    document.write(`First Value: ${value1} (${type1})<br>`);
    document.write(`Second Value: ${value2} (${type2}) → Converted to Number: ${numValue2} (number)<br><br>`);

    document.write(`Using == : ${equalLoose} → Equality ignores type<br>`);
    document.write(`Using === : ${equalStrict} → Strict equality checks type too<br>`);
    document.write(`Using != : ${notEqualLoose} → Values considered equal (ignores type)<br>`);
    document.write(`Using !== : ${notEqualStrict} → Strict inequality because types differ<br>`);
</script>
</body>
</html>
```

4.Lab Question: Bank Customers – Dynamic Array Creation & Manipulation Question: Write a JavaScript program for a bank to store and manage customer names. 1. Ask how many customers to register. 2. Use a dynamic array literal to store their names. 3. Display customer names using a for...of loop. 4. Display customer IDs (array indexes) using a for...in loop. 5. Add a new customer at the end using push(). 6. Remove the last customer using pop(). 7. Insert a new customer at a specific position using splice(). 8. Delete a customer at a specific position using splice() again. 9. Display the final customer list.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Bank Customers Management</title>
</head>
```

```
<body>
<script>
   // Step 1: Ask how many customers to register
   let count = parseInt(prompt("Enter the number of customers to register:"));

   // Step 2: Create a dynamic array literal to store names
   let customers = [];
   for (let i = 0; i < count; i++) {
      let name = prompt(`Enter name of customer ${i + 1}:`);
      customers.push(name);
   }

   // Step 3: Display customer names using for...of
   document.write("<h3>Original Customers: " + customers.join(", ") + "</h3>");
   document.write("<b>Customers using for...of:</b><br>");
   for (let customer of customers) {
      document.write(customer + "<br>");
   }

   // Step 4: Display customer IDs (indexes) using for...in
   document.write("<br><b>Customer IDs using for...in:</b><br>");
   for (let id in customers) {
      document.write(id + "<br>");
   }

   // Step 5: Add a new customer at the end using push()
   customers.push("Anita");
   document.write("<br><b>After push('Anita'):</b> " + customers.join(", ") + "<br>");

   // Step 6: Remove the last customer using pop()
   customers.pop();
```

```
    document.write("<b>After pop():</b> " + customers.join(", ") + "<br>");

    // Step 7: Insert a new customer at a specific position using splice()
    customers.splice(1, 0, "Vikram"); // Insert 'Vikram' at position 1
    document.write("<b>After insert 'Vikram' at position 1:</b> " + customers.join(", ") + "<br>");

    // Step 8: Delete a customer at a specific position using splice() again
    customers.splice(2, 1); // Delete at position 2
    document.write("<b>After delete at position 2:</b> " + customers.join(", ") + "<br>");

    // Step 9: Final customer list
    document.write("<h3>Final Customers: " + customers.join(", ") + "</h3>");
</script>
</body>
</html>
```

5. Lab Question: JSON – Book Store Management Question: Write a JavaScript program to: 1. Create a JSON array of books where each book has: • title (string) • author (string) • price (number) • available (boolean) 2. Display all books using a for...of loop. 3. Display only available books. 4. Add a new book to the JSON array. 5. Update the price of a specific book. 6. Delete a book from the JSON array.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>JSON - Book Store Management</title>
</head>
<body>
<h2>Book Store Management</h2>
<pre id="output"></pre>

<script>
```

```
let output = "";

// 1. Create JSON array of books
let books = [
  { title: "JavaScript Basics", author: "Anu", price: 450, available: true },
  { title: "HTML & CSS", author: "Devi", price: 350, available: false }
];

// 2. Display all books
output += "All Books:\n";
for (let book of books) {
  output += `Title: ${book.title}, Author: ${book.author}, Price: ${book.price}, Available: ${book.available} ₹\n`;
}

// 3. Display only available books
output += "\nAvailable Books:\n";
for (let book of books) {
  if (book.available) {
    output += `Title: ${book.title}, Author: ${book.author}, Price: ${book.price} ₹\n`;
  }
}

// 4. Add a new book
books.push({ title: "Node.js Guide", author: "Karthik", price: 600, available: true });
output += "\nAfter Adding:\n";
output += books.map(book => book.title).join(", ") + "\n";

// 5. Update the price of a specific book
let bookToUpdate = books.find(book => book.title === "JavaScript Basics");
if (bookToUpdate) {
```

```
    bookToUpdate.price = 500;
}
output += "\nAfter Price Update:\n";
output += `JavaScript Basics - ${bookToUpdate.price} ₹\n`;

// 6. Delete a book (remove "JavaScript Basics")
books = books.filter(book => book.title !== "JavaScript Basics");
output += "\nAfter Deletion:\n";
output += books.map(book => book.title).join(", ") + "\n";

// Display output on page
document.getElementById("output").textContent = output;
</script>
</body>
</html>
```

6. Lab Question: Tour Booking Form – getElementById() Question: Write a JavaScript program to: 1. Create a Tour Booking Form with fields: • Full Name (text) • Email (email) • Destination (select) – e.g., Ooty, Goa, Manali • Number of Days (number) • Number of Travelers (number) 2. On clicking "Book Now", • Use getElementById() to read the values. • Calculate total cost based on per-day-per-person price: • Ooty – 2,000/day ₹ • Goa – 3,000/day ₹ • Manali – 4,000/day ₹ • Display a booking summary on the webpage.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Tour Booking Form</title>
</head>
<body>
<h2>Tour Booking Form</h2>

<!-- 1. Form Fields -->
<form id="bookingForm">
```

```
    <label>Full Name: </label>
    <input type="text" id="fullName" required><br><br>

    <label>Email: </label>
    <input type="email" id="email" required><br><br>

    <label>Destination: </label>
    <select id="destination" required>
        <option value="">--Select--</option>
        <option value="Ooty">Ooty</option>
        <option value="Goa">Goa</option>
        <option value="Manali">Manali</option>
    </select><br><br>

    <label>Number of Days: </label>
    <input type="number" id="days" min="1" required><br><br>

    <label>Number of Travelers: </label>
    <input type="number" id="travelers" min="1" required><br><br>

    <button type="button" onclick="bookTour()">Book Now</button>
</form>

<!-- Booking Summary Output -->
<h3>Booking Summary:</h3>
<pre id="summary"></pre>

<script>
function bookTour() {
    // 2. Get form values using getElementById()
    let name = document.getElementById("fullName").value;
```

```
    let email = document.getElementById("email").value;
    let destination = document.getElementById("destination").value;
    let days = parseInt(document.getElementById("days").value);
    let travelers = parseInt(document.getElementById("travelers").value);

    // 3. Price mapping
    let pricePerDay = 0;
    if (destination === "Ooty") {
        pricePerDay = 2000;
    } else if (destination === "Goa") {
        pricePerDay = 3000;
    } else if (destination === "Manali") {
        pricePerDay = 4000;
    }

    // 4. Calculate total cost
    let totalPrice = pricePerDay * days * travelers;

    // 5. Display summary
    let summary = `Name: ${name}
Email: ${email}
Destination: ${destination}
Days: ${days}
Travelers: ${travelers}
Total Price: ${totalPrice} ₹`;

    document.getElementById("summary").textContent = summary;
}
</script>
</body>
</html>
```

Tour Booking Form

Full Name: gayathri busarapu

Email: gayathri.busarapu@gmail.c

Destination: Manali

Number of Days: 9

Number of Travelers: 2

Book Now

Booking Summary:

```
Name: gayathri busarapu
Email: gayathri.busarapu@gmail.com
Destination: Manali
Days: 9
Travelers: 2
Total Price: 72000 ₹
```

7. Lab Question – Sports Team Player Details A sports club stores player information in an object as follows: const player = { name: "Virat Kohli", age: 36, sport: "Cricket", stats: { matches: 500, runs: 25000, average: 57.2 } }; Tasks: 1. Use object destructuring to extract name, sport, and the nested properties matches, runs, and average from player. 2. Using template literals, print the message in this exact format: Player Virat Kohli plays Cricket. Matches: 500 | Runs: 25000 | Batting Average: 57.2 3. Add another player object to an array and use array destructuring to print details for both players in a similar format. Expected Output Example: Player Virat Kohli plays Cricket. Matches: 500 | Runs: 25000 | Batting Average: 57.2 Player Lionel Messi plays Football. Matches: 800 | Goals: 720 | Assists: 300

Code:

```
<!DOCTYPE html>

<html>

<head>

 <title>Sports Team Player Details</title>

</head>

<body>

 <h2>Sports Team Player Details</h2>

 <pre id="output"></pre>


 <script>

  // Player 1

  const player1 = {
```

```
  name: "Virat Kohli",
  age: 36,
  sport: "Cricket",
  stats: {
    matches: 500,
    runs: 25000,
    average: 57.2
  }
};

// Player 2
const player2 = {
  name: "Lionel Messi",
  age: 38,
  sport: "Football",
  stats: {
    matches: 800,
    goals: 720,
    assists: 300
  }
};

// Array of players
const players = [player1, player2];

let result = "";

for (const player of players) {
  // Object destructuring
  const { name, sport, stats } = player;
  const { matches } = stats;
```

```
    // Check if Cricket or Football to handle keys differently
    if (sport === "Cricket") {
      const { runs, average } = stats;
      result += `Player ${name} plays ${sport}.\nMatches: ${matches} | Runs: ${runs} | Batting Average: ${average}\n\n`;
    } else if (sport === "Football") {
      const { goals, assists } = stats;
      result += `Player ${name} plays ${sport}.\nMatches: ${matches} | Goals: ${goals} | Assists: ${assists}\n\n`;
    }
  }

  // Output to webpage
  document.getElementById("output").textContent = result;
 </script>
</body>
</html>
```

8. Lab Question – Switchboard Control (let, var, const) A building has a switchboard to control lights, fans, and AC. You need to write a JavaScript program to simulate turning them ON/OFF using let, var, and const. Tasks: 1. Declare the building name using const (should never change). 2. Declare a let variable to store the status of a device ("ON" or "OFF") — this will change when the switch is pressed. 3. Use var for a loop counter to simulate checking each switch. 4. Inside the loop: • Toggle the device status. • Print a message in this format: Checking Switch 1: Light is ON Checking Switch 2: Fan is OFF Checking Switch 3: AC is ON 5. After the loop, print the var counter to show that it is still accessible outside the loop. 6. Try reassigning the const variable and observe the error.

Code:

```
<!DOCTYPE html>
<html>
<head>
 <title>Switchboard Control</title>
</head>
<body>
```

```
<h2>Switchboard Control Simulation</h2>
<pre id="output"></pre>

<script>
 // 1. Declare building name using const
 const buildingName = "Sunrise Apartments";

 // 2. Declare let variable for device status
 let status = "OFF";

 // Devices in the building
 const devices = ["Light", "Fan", "AC"];

 // Output variable
 let result = "";

 // 3. Use var for loop counter
 for (var i = 0; i < devices.length; i++) {
  // 4. Toggle the device status
  status = (status === "OFF") ? "ON" : "OFF";

  // Print message
  result += `Checking Switch ${i + 1}: ${devices[i]} is ${status}\n`;
 }

 // 5. After the loop
 result += `All switches checked for ${buildingName}.\n`;
 result += `Outside loop: var counter is ${i}\n`;

 // Output to webpage
 document.getElementById("output").textContent = result;
```

```
  // 6. Try reassigning const variable (will throw error in console)
  // buildingName = "Moonlight Residency"; // Uncomment to see error
 </script>
</body>
</html>
```

11. Lab Question – JavaScript Closure (Share Market Example) Objective: Demonstrate the concept of closures in JavaScript by implementing a simple Share Market Portfolio Tracker. Problem Statement: Create a JavaScript program using closures to manage and track a user's share market portfolio. The program should: 1. Allow buying shares (company name, quantity, price per share). 2. Allow selling shares (reduce quantity). 3. Show the total portfolio value. 4. Keep portfolio data private, accessible only via closure functions. • The portfolio array is private — it's not directly accessible from outside the function. • Only buyShare, sellShare, and totalValue have access to it via closure.

Code:

```
<!DOCTYPE html>
<html>
<head>
 <title>Share Market Portfolio Tracker</title>
</head>
<body>
 <h2>Share Market Portfolio Tracker (Closure Example)</h2>
 <pre id="output"></pre>

 <script>
  // Closure function to create a portfolio
  function createPortfolio() {
   let portfolio = []; // Private array (accessible only inside closure)

   function buyShare(company, quantity, price) {
    let existing = portfolio.find(s => s.company === company);
    if (existing) {
     existing.quantity += quantity;
```

```
      existing.price = price; // update latest price
    } else {
      portfolio.push({ company, quantity, price });
    }
    return `${quantity} shares of ${company} bought at ₹${price} each.`;
  }

  function sellShare(company, quantity) {
    let existing = portfolio.find(s => s.company === company);
    if (!existing) return `${company} not found in portfolio.`;
    if (quantity > existing.quantity) return `Not enough shares to sell.`;

    existing.quantity -= quantity;
    if (existing.quantity === 0) {
      portfolio = portfolio.filter(s => s.company !== company);
    }
    return `${quantity} shares of ${company} sold.`;
  }

  function totalValue() {
    let total = portfolio.reduce((sum, share) => sum + (share.quantity * share.price), 0);
    return `Total Portfolio Value: ₹${total}`;
  }

  return { buyShare, sellShare, totalValue };
}

// Using the closure
const myPortfolio = createPortfolio();

let log = "";
```

```
    log += myPortfolio.buyShare("TCS", 10, 3500) + "\n";
    log += myPortfolio.buyShare("Infosys", 5, 1500) + "\n";
    log += myPortfolio.sellShare("TCS", 3) + "\n";
    log += myPortfolio.totalValue() + "\n";

    // Output to page
    document.getElementById("output").textContent = log;

    // Note: portfolio is private
    console.log(myPortfolio.portfolio); // undefined
  </script>
</body>
</html>
```

9.Lab Question: Income Tax Calculator with Multiple Callbacks Question: Write a JavaScript program that: 1. Accepts Name, PAN, and Annual Income from the user. 2. Calculates tax using the slab method. 3. Uses multiple callbacks: • One callback logs the result in the console. • One callback displays the result in the HTML page. • One callback saves the result into a JSON array (like a mini tax record system).

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Income Tax Calculator (Multiple Callbacks)</title>
</head>
<body>
  <h2>Income Tax Calculator (Multiple Callbacks)</h2>
  <div id="result"></div>

  <script>
    // JSON array for storing tax records
    let taxRecords = [];
```

```
// Callback 1 - Log to Console
function logToConsole(details) {
  console.log("Tax Calculation Details:", details);
}

// Callback 2 - Display on Page
function displayOnPage(details) {
  document.getElementById("result").innerHTML = `
    <b>Name:</b> ${details.name} <br>
    <b>PAN:</b> ${details.pan} <br>
    <b>Annual Income:</b> ₹ ${details.income} <br>
    <b>Calculated Tax:</b> ₹ ${details.tax}
  `;
}

// Callback 3 - Save to Records
function saveToRecords(details) {
  taxRecords.push(details);
  console.log("Tax Records Updated:", taxRecords);
}

// Main tax calculation function
function calculateTax(name, pan, income, callback) {
  let tax = 0;

  // Simple slab method
  if (income <= 250000) tax = 0;
  else if (income <= 500000) tax = (income - 250000) * 0.05;
  else if (income <= 1000000) tax = (250000 * 0.05) + ((income - 500000) * 0.2);
  else tax = (250000 * 0.05) + (500000 * 0.2) + ((income - 1000000) * 0.3);
```

```
    const details = {
      name: name,
      pan: pan,
      income: income,
      tax: tax
    };

    // Execute the callback with details
    callback(details);
  }

  // Multiple callbacks execution
  function processTaxCalculation(details) {
    logToConsole(details);
    displayOnPage(details);
    saveToRecords(details);
  }

  // Example execution
  calculateTax("Arjun R", "ABCDE1234F", 750000, processTaxCalculation);
 </script>
</body>
</html>
```

10. Build a multi-section RTO Vehicle Registration form with Bootstrap that validates: • Owner: full name, email, mobile (India), DOB (≥18 yrs), Aadhaar, PAN. • Address: address lines, city, state (select), PIN (6 digits). • Vehicle: registration no., class (select), fuel (radio), make/model, chassis no., engine no., year of manufacture, purchase date (not future), color. • Compliance: emission norm (BS-III…BS-VI), insurance policy no., insurer name, insurance expiry (≥ today), PUC valid till (≥ today). • Finance (optional): hypothecation (checkbox), if yes collect bank/NBFC, agreement date. • Uploads: ID/address proof, insurance copy (PDF/JPG/PNG ≤ 2 MB), optional vehicle image (≤ 2 MB). • Declarations: terms checkbox + simple math CAPTCHA. UX requirements: • Sections grouped in a Bootstrap Accordion, show a top progress bar that updates as sections validate. • Use Bootstrap validation styles (is-valid / is-invalid). • Custom messages for common errors. • Disable Submit until all required fields are valid + terms checked + captcha correct. • On success, show a Bootstrap toast "Application submitted".

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>RTO Vehicle Registration Form</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background: #f8f9fa;
    }
    .container {
      background: white;
      padding: 20px;
      border-radius: 8px;
      margin-top: 20px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }
    h4 {
      margin-top: 20px;
      padding-bottom: 5px;
      border-bottom: 2px solid #dee2e6;
    }
  </style>
</head>
<body>

<div class="container">
  <h2 class="text-center mb-4">RTO Vehicle Registration Form</h2>
```

```
<form id="rtoForm">

  <!-- Owner Section -->
  <h4>Owner Details</h4>
  <div class="mb-3">
    <label>Full Name</label>
    <input type="text" class="form-control" required>
  </div>
  <div class="mb-3">
    <label>Email</label>
    <input type="email" class="form-control" required>
  </div>
  <div class="mb-3">
    <label>Mobile (India)</label>
    <input type="tel" pattern="[6-9]{1}[0-9]{9}" class="form-control" required>
  </div>
  <div class="mb-3">
    <label>Date of Birth</label>
    <input type="date" class="form-control" required>
  </div>
  <div class="mb-3">
    <label>Aadhaar Number</label>
    <input type="text" pattern="\d{12}" class="form-control" required>
  </div>
  <div class="mb-3">
    <label>PAN Number</label>
    <input type="text" pattern="[A-Z]{5}[0-9]{4}[A-Z]{1}" class="form-control" required>
  </div>

  <!-- Address Section -->
  <h4>Address</h4>
```

```
<div class="mb-3">
  <label>Address Line 1</label>
  <input type="text" class="form-control" required>
</div>
<div class="mb-3">
  <label>Address Line 2</label>
  <input type="text" class="form-control">
</div>
<div class="mb-3">
  <label>City</label>
  <input type="text" class="form-control" required>
</div>
<div class="mb-3">
  <label>State</label>
  <select class="form-control" required>
    <option value="">Choose...</option>
    <option>Andhra Pradesh</option>
    <option>Telangana</option>
    <option>Tamil Nadu</option>
    <option>Karnataka</option>
    <option>Maharashtra</option>
  </select>
</div>
<div class="mb-3">
  <label>PIN Code</label>
  <input type="text" pattern="\d{6}" class="form-control" required>
</div>

<!-- Vehicle Section -->
<h4>Vehicle Details</h4>
<div class="mb-3">
```

```
    <label>Registration Number</label>
    <input type="text" class="form-control" required>
</div>
<div class="mb-3">
    <label>Vehicle Class</label>
    <select class="form-control" required>
        <option value="">Choose...</option>
        <option>Two Wheeler</option>
        <option>Four Wheeler</option>
        <option>Commercial</option>
    </select>
</div>
<div class="mb-3">
    <label>Fuel Type</label><br>
    <input type="radio" name="fuel" value="Petrol" required> Petrol
    <input type="radio" name="fuel" value="Diesel"> Diesel
    <input type="radio" name="fuel" value="Electric"> Electric
</div>
<div class="mb-3">
    <label>Make/Model</label>
    <input type="text" class="form-control" required>
</div>
<div class="mb-3">
    <label>Chassis Number</label>
    <input type="text" class="form-control" required>
</div>
<div class="mb-3">
    <label>Engine Number</label>
    <input type="text" class="form-control" required>
</div>
<div class="mb-3">
```

```
    <label>Year of Manufacture</label>
    <input type="number" min="1900" max="2025" class="form-control" required>
</div>
<div class="mb-3">
    <label>Purchase Date</label>
    <input type="date" class="form-control" required>
</div>
<div class="mb-3">
    <label>Color</label>
    <input type="text" class="form-control" required>
</div>

<!-- Compliance Section -->
<h4>Compliance & Insurance</h4>
<div class="mb-3">
    <label>Emission Norm</label>
    <select class="form-control" required>
        <option>BS-III</option>
        <option>BS-IV</option>
        <option>BS-VI</option>
    </select>
</div>
<div class="mb-3">
    <label>Insurance Policy Number</label>
    <input type="text" class="form-control" required>
</div>
<div class="mb-3">
    <label>Insurer Name</label>
    <input type="text" class="form-control" required>
</div>
<div class="mb-3">
```

```
    <label>Insurance Expiry</label>
    <input type="date" class="form-control" required>
</div>
<div class="mb-3">
    <label>PUC Valid Till</label>
    <input type="date" class="form-control" required>
</div>

<!-- Finance Section -->
<h4>Finance (Optional)</h4>
<div class="mb-3 form-check">
    <input type="checkbox" id="hypothecation" class="form-check-input">
    <label for="hypothecation">Hypothecation</label>
</div>
<div class="mb-3">
    <label>Bank/NBFC Name</label>
    <input type="text" class="form-control">
</div>
<div class="mb-3">
    <label>Agreement Date</label>
    <input type="date" class="form-control">
</div>

<!-- Uploads -->
<h4>Uploads</h4>
<div class="mb-3">
    <label>ID/Address Proof</label>
    <input type="file" class="form-control" accept=".pdf,.jpg,.jpeg,.png" required>
</div>
<div class="mb-3">
    <label>Insurance Copy</label>
```

```
      <input type="file" class="form-control" accept=".pdf,.jpg,.jpeg,.png" required>
    </div>
    <div class="mb-3">
      <label>Vehicle Image (Optional)</label>
      <input type="file" class="form-control" accept=".jpg,.jpeg,.png">
    </div>

    <!-- Declaration -->
    <h4>Declaration</h4>
    <div class="mb-3 form-check">
      <input type="checkbox" id="terms" class="form-check-input" required>
      <label for="terms">I agree to the terms and conditions</label>
    </div>

    <div class="mb-3">
      <label>CAPTCHA: 5 + 3 = ?</label>
      <input type="number" class="form-control" required>
    </div>

    <button type="submit" class="btn btn-primary w-100">Submit Application</button>
  </form>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```